BOOKBRIDGE

Community Book Exchange & Lending Platform

Read More. Own Less. Share Books.

Tech Stack: React.js • Java • Spring Boot • Maven • MySQL • Spring Data JPA / Hibernate

Field	Detail
Project	BOOKBRIDGE
Application type	Community book exchange and peer-to-peer lending platform
Required stack	React.js • Java • Spring Boot • Maven • MySQL • Spring Data JPA / Hibernate
Architecture	React.js → Spring Boot REST API → Java service layer → JPA/Hibernate → MySQL
Name	Yeldhandi Chandu
Roll No.	24EG105P62
Branch & Section	CSE-P

1. Problem Statement

Sharing and re-selling personal books inside a college, apartment complex, or neighbourhood is usually handled through scattered WhatsApp groups, notice-board posts, and word of mouth. This makes it hard to know what is actually available, who currently holds a book, and whether it will come back.

- Fragmented discovery — Books available to borrow are scattered across chats, groups, and shelves instead of one searchable place.
- No borrowing record — There is no shared record of who borrowed which book, when it is due, and whether it was returned.
- Trust gap — New members cannot tell whether an owner or borrower has a reliable track record before agreeing to an exchange.
- Duplicate or conflicting requests — The same book can be promised to more than one requester because there is no single source of truth.
- Idle personal libraries — Books that could be shared sit unused because there is no simple way to list, request, and return them.

2. Proposed Solution

BookBridge is a real full-stack web application that lets community members list personal books, discover books nearby, request to borrow, and track the full lending lifecycle from request to return.

Core proposition: a single relational system links members, books, borrow requests, and reviews so that every dashboard number, book status, and rating is derived from backend data rather than hardcoded values.

3. Objectives, Scope, and User Roles

The objective is to make informal book-sharing safe, searchable, and accountable. BookBridge covers the complete lifecycle from account registration and book listing through borrow requests, approval, return, and rating. Every member can act both as an owner (listing books) and as a borrower (requesting books), so a single MEMBER role keeps the system simple.

3.1 Member capabilities as an owner

- Register and log in with a secure password-hashed account; manage name, email, phone, and location.
- List a personal book with title, author, genre, condition, and description; edit or remove a listing.
- Review incoming borrow requests and approve or reject each one.
- Mark an approved loan as returned once the book physically comes back.

3.2 Member capabilities as a borrower

- Browse, search, and filter books by title/author, genre, location, and availability.
- Open a book's detail page and send a borrow request with a proposed due date.
- Track sent requests, current borrowings, borrowing history, and due dates.
- Leave a rating and short comment for the owner after a completed and returned loan.

3.3 Vocabulary and lifecycle

Category	Values
Genres	Fiction; Non-Fiction; Academic; Children; Biography; Self-Help; Other
Book condition	NEW; GOOD; FAIR; WORN
Book status	AVAILABLE; REQUESTED; BORROWED; UNAVAILABLE
Request status	PENDING; APPROVED; REJECTED; RETURNED; CANCELLED

4. Authentication and Profile

Registration collects Full Name, Email, Password, Phone Number, and Location. There is a single MEMBER role. Login uses email and password. Passwords are never stored in plain text; the Spring Boot backend applies secure password hashing.

Endpoint	Purpose
POST /api/auth/register	Create a member account
POST /api/auth/login	Authenticate and issue a session/token
GET /api/users/{id}	Return a member profile
PUT /api/users/{id}	Update profile fields
GET /api/users/{id}/statistics	Return derived borrowing and lending statistics

A member profile presents name, email, phone, location, books listed, books currently lent out, books currently borrowed, completed exchanges, and average rating received as an owner and as a borrower.

5. Book Listing and Discovery

Owners create persistent MySQL book records through the Spring Boot REST API. The React discovery page requests book data from the backend; it does not use hardcoded cards or statistics.

5.1 Book fields

Field group	Required information
Identity	Title, author, genre, owner
Condition	Condition rating, description/notes
Availability	Location, current status
Lifecycle	AVAILABLE, REQUESTED, BORROWED, or UNAVAILABLE status

5.2 Discovery and book details

- Book cards show title, author, genre, condition, location, owner, status, and a Request to Borrow button.
- Search matches title or author. Filters include genre, location, and availability status.
- The detail page shows description, condition, owner profile summary, current status, and owner rating.
- A successful request changes the book's action to Requested. A book already REQUESTED or BORROWED cannot receive a second active request.

5.3 Ownership and request relationship

A member can own many books, and a member can send many borrow requests. Each book belongs to exactly one owner; each borrow request links exactly one book to exactly one requester and carries its own status, request date, and due date.

Endpoint	Purpose
GET /api/books	List books with search and filters
GET /api/books/{id}	Return book details and current status
POST /api/books	Create an owner-listed book
PUT /api/books/{id}	Edit book fields or status
DELETE /api/books/{id}	Remove a listing when it has no active request
POST /api/books/{bookId}/request	Send a borrow request for the book

Availability rule: the Java service checks for an existing PENDING or APPROVED request before accepting a new one. It rejects a request on an unavailable book, rejects a self-request from the book's own owner, and returns a meaningful validation error to React.

6. Borrowing, Returns, and Ratings

BookBridge treats every loan as a tracked record, not an informal hand-off. Only RETURNED loans are eligible for rating, and only submitted ratings contribute to a member's average score.

6.1 Request approval and return

- Owners see incoming requests with requester name, proposed due date, and request date.
- Only the owning member can approve or reject a request for their own book.
- An approved request moves the book to BORROWED; a rejected request returns the book to AVAILABLE.
- The owner marks a loan as RETURNED once the book comes back, which returns the book to AVAILABLE and unlocks rating.

6.2 Reviews and ratings

Field	Purpose
Borrow request reference	Links the review to exactly one completed loan
Reviewer / reviewee	Borrower rates the owner, or the owner rates the borrower
Rating	1 to 5 scale
Comment	Optional short note on the exchange

Each completed loan allows at most one review per direction (borrower → owner and owner → borrower). The service prevents a second review from being submitted for the same request and direction.

6.3 Dashboard statistics

Owner view	Borrower view
Books listed; books currently lent out; pending incoming requests; average rating received	Books currently borrowed; borrowing history; pending sent requests; average rating received

All counts, statuses, and average ratings are queried or calculated from backend/database records. No dashboard statistic is hardcoded.

7. System Architecture

The application uses a layered architecture. React.js communicates with the Spring Boot REST API; controllers delegate to Java services; services use Spring Data JPA and Hibernate repositories; MySQL provides durable relational storage.



Figure 1 — Required BookBridge request architecture.

8. End-to-End Request Flow

A request never bypasses the service and persistence layers. For example, sending a borrow request checks the book's current status in MySQL, applies availability and ownership rules in Java, and returns a JSON response for React to render.

React UI *user action*	REST API *HTTP request*	Controller *route + DTO*	Service *rules + validation*	Repository *JPA query*	MySQL *read / write*

Figure 2 — One request travels downward through the layers; the response returns to React.

8.1 Cross-cutting controls

- Bean Validation checks required fields, valid enumerated statuses, and due-date consistency.
- Ownership-based authorization restricts book editing, approval, and return actions to the book's owner.
- DTOs separate API payloads from entities; exception handling returns consistent validation and not-found responses.
- Hibernate maps JPA relationships and foreign keys; repository queries supply book lists, requests, statistics, and reviews.

8.2 Frontend responsibility

React.js provides the responsive single-page interface and REST API integration. Its screens include the landing page, registration/login, dashboard, profile, discovery, book details, book listing/editing, requests, and reviews. The frontend renders live API responses rather than duplicating backend rules.

9. Home Page and Frontend Flows

The landing page introduces BookBridge with the tagline “Read More. Own Less. Share Books.” and the description: “List the books gathering dust on your shelf, discover what your neighbours are lending, and keep every exchange on record.”

Landing section	Required behavior
Hero	BookBridge title, tagline, description, Browse Books, List a Book, and Join Now buttons
How It Works	List → Discover → Request → Approve → Exchange → Rate
Featured Books	Real recently listed, available book records returned from MySQL through the API
Community Snapshot	Real totals: total books listed, active members, books exchanged, average rating

10. Owner Flow

Register / Login *create member account*	List Book *title, genre, condition*	Review Requests *approve or reject*	Hand Over Book *status becomes BORROWED*	Mark Returned *status becomes AVAILABLE*	Rate Borrower *optional feedback*

Figure 3 — Owner journey from listing a book to a completed, rated exchange.

11. Borrower Flow

Discover Books	Review Details	Send Request	Get Approved	Return Book	Rate Owner
search and filter	*condition, owner, location*	*proposed due date*	*book marked BORROWED*	*owner confirms return*	*optional feedback*

Figure 4 — Borrower journey from discovery to a completed, rated exchange.

11.1 Responsive screen organization

The React navigation keeps everyday actions close at hand: Dashboard, Discover Books, My Listings, My Requests, Profile, and Reviews are reachable by every member, since every member can act as both owner and borrower.

12. Spring Boot Backend Structure

The backend is a Maven project organized as a clean layered Spring Boot application. Important application logic is implemented in Java service classes rather than in React or database triggers.

```
src/main/java/com/bookbridge
|-- controller   AuthController, UserController, BookController,
|                 BorrowRequestController, ReviewController
|-- service      AuthService, UserService, BookService,
|                 BorrowRequestService, ReviewService
|-- repository   UserRepository, BookRepository,
|                 BorrowRequestRepository, ReviewRepository
|-- entity       User, Book, BorrowRequest, Review
|-- dto          request and response payloads
|-- exception    validation, not-found, authorization handlers
`-- config       security, password hashing
```

12.1 REST API catalogue

Area	Representative endpoints
Authentication	POST /api/auth/register; POST /api/auth/login
Users	GET /api/users/{id}; PUT /api/users/{id}; GET /api/users/{id}/statistics
Books	GET /api/books; GET /api/books/{id}; POST /api/books; PUT /api/books/{id}; DELETE /api/books/{id}
Borrow Requests	POST /api/books/{bookId}/request; PUT /api/requests/{id}/approve; PUT /api/requests/{id}/reject; PUT /api/requests/{id}/return; GET /api/users/{id}/requests
Reviews	POST /api/requests/{id}/review; GET /api/users/{id}/reviews

13. Java Business Rules

Rule	Service-layer behavior
Registration	Reject duplicate email; hash password; create a single MEMBER account.
Authorization	Only a book's owner can edit, approve, reject, or mark it returned.
Self-request	Reject a borrow request where the requester is also the book's owner.
Availability	Reject a new request when the book already has a PENDING or APPROVED request.
Approval	Approving one request for a book auto-rejects any other pending request for that book.
Return	Only an APPROVED request can be marked RETURNED; returning frees the book to AVAILABLE.
Reviews	Allow at most one review per direction per completed request; average rating recalculated after each review.

Maven must include a proper pom.xml with Spring Boot, Spring Web, Spring Data JPA, Hibernate, Bean Validation, and the MySQL connector. Maven compiles, tests, and packages the backend as a runnable application.

14. Database Design — MySQL

MySQL is the main relational database. The schema is normalized and uses foreign keys to avoid duplicating member and book data. The following tables are required for the platform's core records.

Table	Key fields and purpose
users	id PK; name; email UNIQUE; password hash; phone; location; created_at
books	id PK; title; author; genre; condition; description; location; status; owner_id FK; created_at
borrow_requests	id PK; book_id FK; requester_id FK; request_date; due_date; return_date; status
reviews	id PK; borrow_request_id FK; reviewer_id FK; reviewee_id FK; rating; comment; created_at

14.1 Relationships and integrity

- users → books is one-to-many through owner_id: a member lists many books.
- users → borrow_requests is one-to-many through requester_id: a member sends many requests.
- books → borrow_requests is one-to-many: a book can accumulate a history of requests over time, though only one may be active at a time.
- borrow_requests → reviews is one-to-many (at most two: one per direction): a completed loan can carry a borrower review and an owner review.

14.2 Representative relational constraints

Constraint	Reason
users.email UNIQUE	One login identity per email address
Unique (borrow_request_id, reviewer_id) on reviews	Prevents a duplicate review from the same party for the same loan
Foreign keys with appropriate delete/update policy	Keeps requests and reviews linked to valid parent rows
NOT NULL and range validation	Protects required names, dates, ratings (1–5), and status values

The database stores records; Java services enforce the business decisions — availability, ownership, and single-active-request rules — before changes are committed.

15. Entity Relationship Diagram

The ER model below covers the primary BookBridge records and their cardinalities.

Entity	Responsibility
User	Identity, contact details, and location of a community member
Book	Listed title, genre, condition, owner, and current lifecycle status
BorrowRequest	Book-requester link with request date, due date, return date, and status
Review	Post-loan rating and comment between borrower and owner

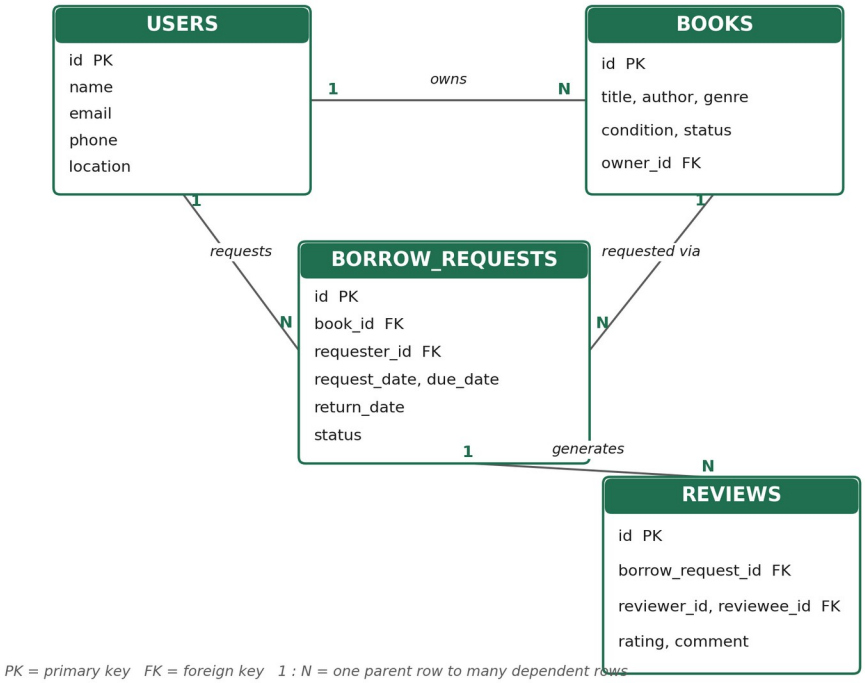


Figure 5 — Simplified ER model for members, books, borrow requests, and reviews.

16. Data-to-UI Reporting

A returned loan and its review can be displayed in three places without copying the data itself: the book's history, the owner's profile, and the borrower's profile. Each view reads the same MySQL-backed record through appropriate API responses.

Borrower Requests	Owner Approves	Book Returned	Reviews Submitted	Profiles Updated
PENDING	*APPROVED*	*RETURNED*	*rating stored*	*averages recalculated*

Figure 6 — Loan lifecycle and rating path.

Integrity principle: a pending or rejected request must not affect a book's public availability history, a member's completed-exchange count, or their average rating.

17. Core User Flows and Implementation Checklist

The following flows summarize the complete product behavior and provide a concise acceptance checklist for the full-stack implementation.

17.1 Owner flow

Register	List Book	Approve/Reject	Return	Reputation
create account	*title, genre, condition*	*incoming requests*	*confirm hand-back*	*rating updates*

Figure 7 — Owner flow from listing to a completed, rated exchange.

17.2 Borrower flow

Discover	Request	Borrow	Return	Reputation
search + filters	*propose due date*	*book handed over*	*confirmed by owner*	*rating updates*

Figure 8 — Borrower flow from discovery to a completed, rated exchange.

Requirement area	Compliance statement
Frontend	React.js, JavaScript, responsive HTML/CSS through React components, REST API integration.
Backend	Java Spring Boot, Maven, Spring Web/REST, Spring Data JPA, Hibernate, Bean Validation.
Database	MySQL main database with normalized tables and foreign-key relationships.
Security	Secure password hashing and ownership-based authorization for listing and request actions.
Core behavior	Search/filter books, single-active-request rule, request approval, return confirmation.
Reviews	Post-loan ratings limited to one per direction per completed request.
UI coverage	Landing page, dashboard, profile, discovery, book details, listing management, requests, reviews.

18. Conclusion

BookBridge turns informal book-sharing into a traceable, accountable community activity. Its React interface keeps listing and borrowing simple, its Spring Boot service layer enforces ownership and availability rules, and its MySQL model preserves a reliable history of members, books, loans, and ratings. The design is intentionally small in scope, fully relational, and well suited for implementation as a B.Tech CSE full-stack project.

BookBridge — Read More. Own Less. Share Books.